

A Multi-Agent System for Solving Math Problems Verified with Lean 4

Juan Pablo Dueñas Becerra, Julian Andres Velandia Serrate
Universidad Pedagógica y Tecnológica de Colombia, Sogamoso, Colombia
juan.duenas@uptc.edu.co, julian.velandia01@uptc.edu.co

Abstract

In this project we are building a system of AI agents that solves olympiad math problems, and the only way a problem counts as solved is if the Lean 4 kernel accepts the proof, with no `sorry` and no extra axioms added by hand. The idea comes from the fact that a language model can write proofs that look fine but are wrong, and on top of that it says they are correct, so we keep generation flexible and acceptance strict. The system has six agents (**Planner**, **Retriever**, **Generator**, **Autoformalizer**, **Verifier** and **Critic**) organized as a state graph, and the Verifier is not a model but a deterministic component around the kernel. By week 8 we have the verification environment, the Verifier with seven verdicts, the miniF2F data, the baselines, the Planner with a verified skeleton, an index of 179,269 Mathlib lemmas (recall@k of 0.173, 0.224 and 0.287 for k = 5, 10 and 20) and the full graph, which already solved a real problem end to end with the real kernel. Testing against real Lean showed six bugs we could not see before. To avoid paying for each call, we moved the language models to Qwen3-4B running locally with Ollama.

Keywords—*automated theorem proving, Lean 4, multi-agent systems, language models, formal verification, local models.*

I. INTRODUCTION

Preface. Language models have become much better at math, but what they produce is text that sounds right, not something you can check. They can reach the right answer with a wrong step in the middle, or give a bad proof and say very confidently that it is correct. Asking them to reason step by step helps [9], but it does not guarantee anything. Proof assistants like Lean 4 [1] work from the other side: the kernel checks every step and says clearly if the proof is valid or not. What we do is put the two together: the models generate and the kernel accepts. **A problem is solved if and only if the kernel accepts the assembled proof, with no `sorry` and no axioms outside the three core axioms of Lean.**

Short background. Draft, Sketch, and Prove [4] showed that an informal sketch helps a formal prover, LeanDojo [5] that retrieving Mathlib lemmas raises the success rate, and Baldur [6] that the compiler error is useful to fix proofs. Today DeepSeek-Prover [7] and Goedel-Prover-V2 [8] are the state of the art on miniF2F [3].

Goal and contribution. We want to build and test a multi-agent system where every step of the decomposition can be checked mechanically. We are not trying to beat the state of the art. What we want to add are three things we did not see together in other works: checking the skeleton before proving anything, working with a fixed budget of 20 calls per problem, and measuring how many times the model says its proof is right and the kernel says it is not.

Who can benefit. Researchers in AI and automated reasoning, because the architecture, contracts and logs can be reused; math teachers and students, who get proofs checked step by step; the Lean and Mathlib community, because of the Autoformalizer and the Retriever; and groups with few resources, because everything runs locally with no cost per call on a 4 GB GPU.

Scope. The schedule is 13 weeks and this report covers weeks 1 to 8. We work with the miniF2F problems (they are already formalized), one fixed version of Lean and Mathlib, local models and the budget of 20 calls. Formalizing problems written in natural language, training models, or competing with systems that use big models through an API are out of scope. Weeks 9 to 13 are for the Critic, the ablation, the analysis and the final report.

Organization. Section II has the background and how it connects to our work; Section III (Materials and methods) has the concepts, data, architecture, agents, state model, connection to Lean, math model, local models and experimental design; Section IV has the results, and Sections V and VI the conclusions and next steps.

II. BACKGROUND

We use each previous work to support some design decision (Table I). Chain-of-Thought [9] and self-consistency [10] are behind the Generator; ReAct [11] and Reflexion [12] are behind the formalize, verify and correct loop and the Critic, with the difference that here the one who observes and criticizes is the kernel, not the model itself. RAG [13], DPR [14], BGE-M3 [15] and FAISS [16] are the base of the Retriever. Lean 4 [1] and Mathlib [2] are the environment, autoformalization [17] is the task of the Autoformalizer, and the work closest to

ours is Draft, Sketch, and Prove [4], to which we add checking the skeleton before spending calls. We use miniF2F [3] as the benchmark and PutnamBench [18] as a second one, and Qwen3 [19] with Ollama [20] to run everything locally.

TABLE I
BACKGROUND AND HOW IT CONNECTS TO THE SYSTEM

Previous work	What we take	Where it is in the system
Chain-of-Thought [9], self-consistency [10]	step by step reasoning; several samples and a vote	Generator, Eq. 5–6
ReAct [11], Reflexion [12], Baldur [6]	act, observe and fix using the error	state graph, Critic
RAG [13], DPR [14], BGE-M3 [15], FAISS [16]	retrieve before generating	Retriever, Eq. 3–4
LeanDojo [5]	Mathlib premises	our own corpus from the same commit
Draft, Sketch, and Prove [4]	sketch $\rightarrow$ skeleton $\rightarrow$ holes	Planner + VERIFY_SKELETON
Goedel-Prover-V2 [8]	specialized prover	Autoformalizer, baseline A
miniF2F [3]	formalized problems	development and test data
Qwen3 [19], Ollama [20]	local model, zero cost	Planner, Generator, Critic

III. MATERIALS AND METHODS

A. Basic concepts

- **Lean 4, kernel and Mathlib:** the *kernel* is the part of Lean that checks that every step of a proof is valid, and Mathlib is Lean's math library, with hundreds of thousands of theorems already proved.
- **REPL (Read–Eval–Print Loop):** a program that stays open, receives code, runs it and returns the result. We use it because loading Mathlib takes tens of seconds, and this way it only loads once.
- **miniF2F:** 488 olympiad problems that someone already translated to Lean and reviewed, split in two halves of 244: *development* for tuning and *test*, which is opened only once at the end.
- **Skeleton with sorry:** in Lean, *sorry* is like saying "I will prove this later". The Planner writes the theorem split into steps that end in *sorry* (Fig. 4a), and if that compiles, Lean already checked that the steps lead to the theorem. After that, each *sorry* is replaced by a real proof.
- **Ablation:** removing one part of the system at a time and seeing how much the performance drops, to know what each part adds.

B. Data source

We use miniF2F [3] in the Lean 4 version published by DeepSeek-Prover [7], with 244 development problems and 244 test problems, and we check it with its SHA-256 hash. The test set stays sealed until week 11. For the Retriever we took a corpus of 252,002 theorems from the same Mathlib commit the Verifier uses (9837ca9d, v4.15.0), which goes down to 179,269 after removing the ones Lean generates automatically.

C. System architecture

Fig. 1 shows the architecture in three parts: at the top the miniF2F problem comes in, in the middle the LangGraph orchestrator coordinates the six agents with a limit of 20 calls per problem, and at the bottom are the local resources each agent uses. Every call to a model goes through one single function that takes it from the budget and writes a JSONL line with tokens, latency and cost, and all the metrics come from there.

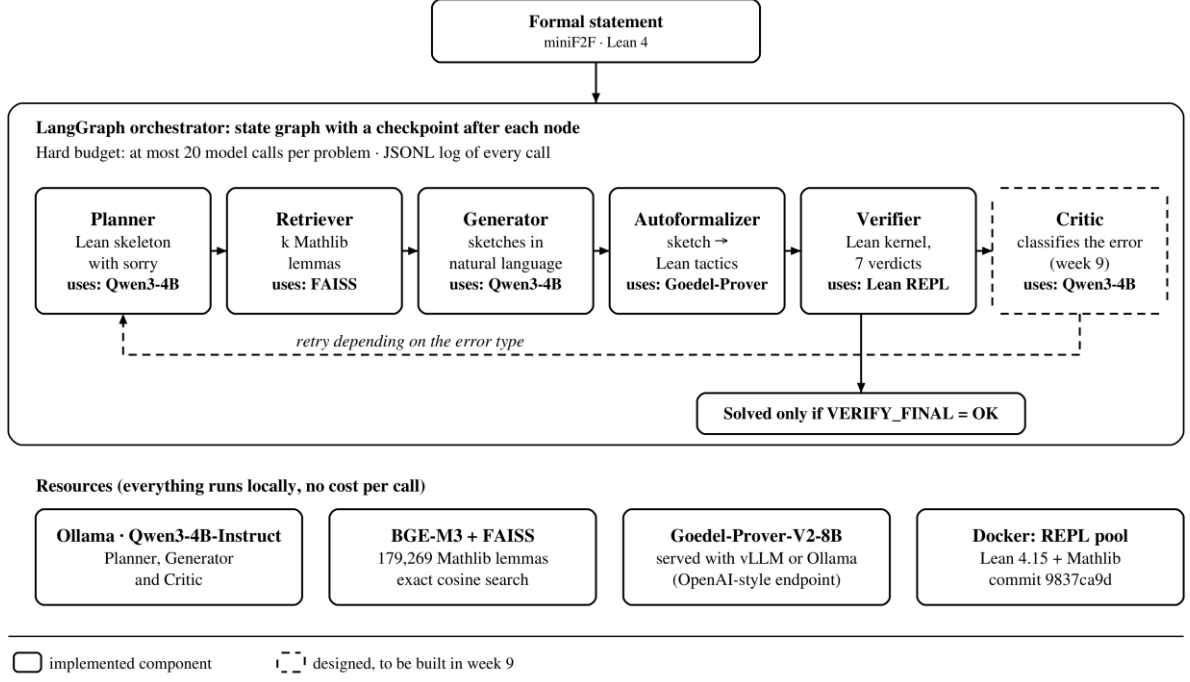


Fig. 1. System architecture. Each agent shows the resource it uses. The Critic has a dashed border because it is already designed but will be built in week 9.

D. Agents

Each agent has a JSON Schema contract for its output and its prompt saved as a versioned file (Table II).

TABLE II
SYSTEM AGENTS

Agent	Type	Input	Output	Status
Planner	language model (Qwen3-4B)	problem	Lean skeleton with sorry	done
Retriever	similarity search	one step	k Mathlib lemmas	done
Generator	language model (Qwen3-4B)	step + lemmas	2 sketches and the chosen one	done
Autoformalizer	prover (Goedel-Prover-V2-8B)	sketch	Lean tactics	done
Verifier	deterministic (Lean kernel)	Lean code	1 of 7 verdicts	done
Critic	language model (Qwen3-4B)	verdict + error	retry action	week 9

We need to make clear that **the Verifier is not a language model**. It is a deterministic component around the kernel, and the same code always gets the same verdict. We treat it as an agent because it sits on the same boundary as the others (input and output through a contract) and because its verdict decides where the graph goes next; also, it is the only one that can say a problem is solved. **We keep the Critic as part of the design** even though it is not built yet: its contract, prompt and transitions are already defined, and the ablation needs it to compare against the version without a Critic.

E. State model

Fig. 2 shows the graph. We added VERIFY_SKELETON to the original proposal, which checks the decomposition before spending calls, and the pair ASSEMBLE / VERIFY_FINAL, because one step compiling on its own does not mean the whole theorem compiles. Without a Critic, when a step fails it is retried blindly (two times max), so the ablation measures what reading the error adds and not just trying again.

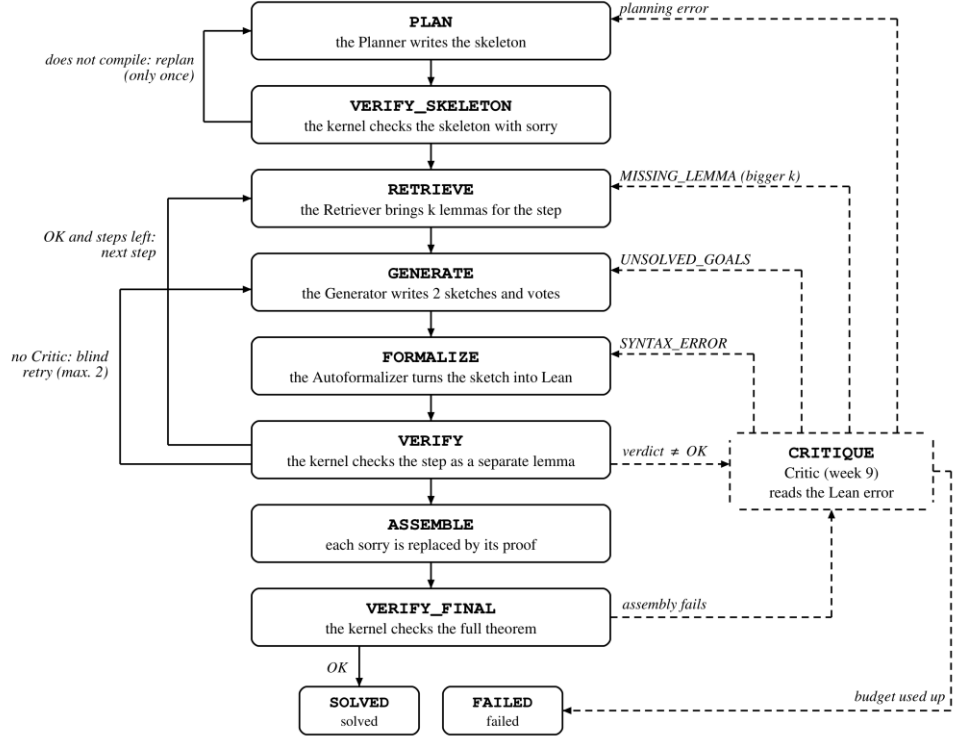


Fig. 2. State graph. Dashed arrows are the Critic transitions (week 9) depending on the verdict; on the left are the next step and the blind retry without a Critic.

F. Verification environment and connection to Lean

The environment is a Docker image with Lean 4.15, Mathlib already compiled and the community REPL. Python starts the REPL as a subprocess (`lake env repl`) and sends it a JSON line through standard input, `{"cmd": code, "env": n}`, and the REPL answers with the errors, the remaining sorry and an environment number (Fig. 3). The miniF2F header is compiled once and then reused. Before sending anything we check that the problem statement did not change, and if it compiles we ask for the axioms with `#print axioms`. With a pool of REPLs we can verify in parallel. At the end we get one of seven verdicts (Table III), and SORRY and AXIOM are there so that a hole or a made-up axiom does not count as a proof.

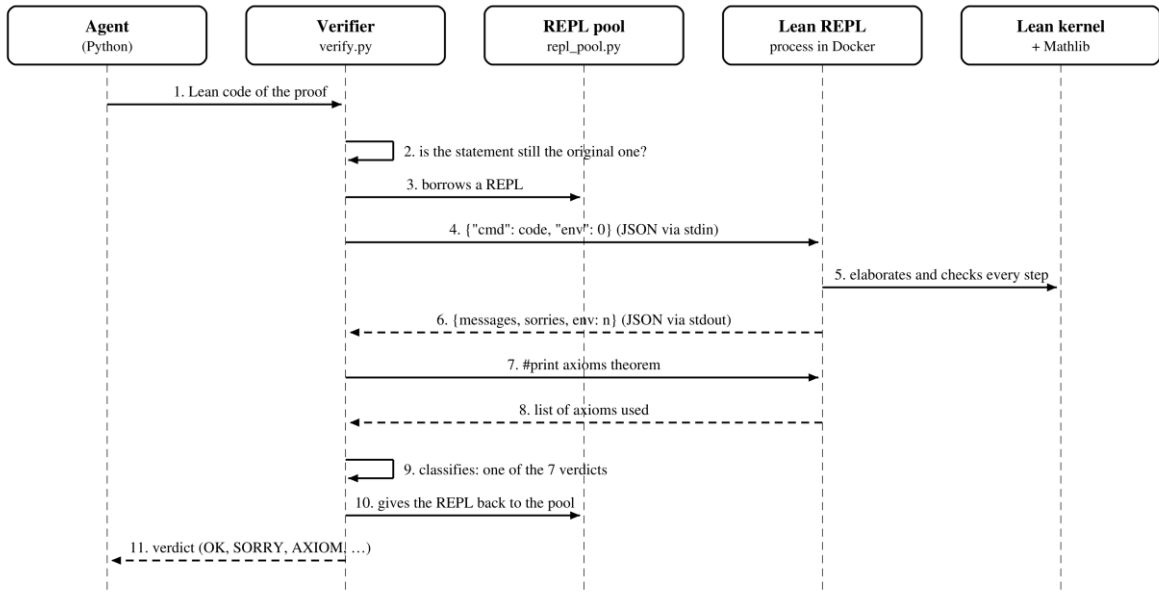


Fig. 3. Sequence of one verification. Solid arrows are requests and dashed arrows are responses.

TABLE III
VERIFIER VERDICTS

Verdict	When it happens	What it is for
OK	compiles, no sorry, allowed axioms	only way to solved
SORRY	compiles but sorry remains	so a hole does not count as a proof
AXIOM	uses axioms outside the core	blocks tricks like axiom ... : False
SYNTAX_ERROR	parse or elaboration error	cheap retry
MISSING_LEMMA	unknown identifier	made-up lemma: signal for the Retriever
UNSOLVED_GOALS	goals not closed	the sketch is wrong, not the syntax
TIMEOUT	time runs out or the process dies	tactics that never finish

G. Math model

Here are the equations the system uses, each one with its name, where it is used and what it means in plain words. Table IV summarizes them.

Equation 1: success criterion (indicator function). Used in VERIFY_FINAL.

$$\text{solved}(p) = 1 \quad [V(\text{assemble}(p)) = \text{OK}] \quad (1)$$

In plain words: the problem is worth 1 only if, when the proofs of all its steps are put together, the Verifier V says OK. This way nobody except the kernel can say that something was solved.

Equation 2: skeleton rule. Used in VERIFY_SKELETON; H are the hypotheses, s_i the steps and T the conclusion.

$$(H, s_1, \dots, s_n \vdash T) \wedge \forall_i (H \vdash s_i) \Rightarrow H \vdash T \quad (2)$$

In plain words: if the steps lead to the conclusion (Lean checks this when it compiles the skeleton) and each step can be proved on its own, the theorem is proved. With this, the decomposition stops being a hope and can be checked (Fig. 4).

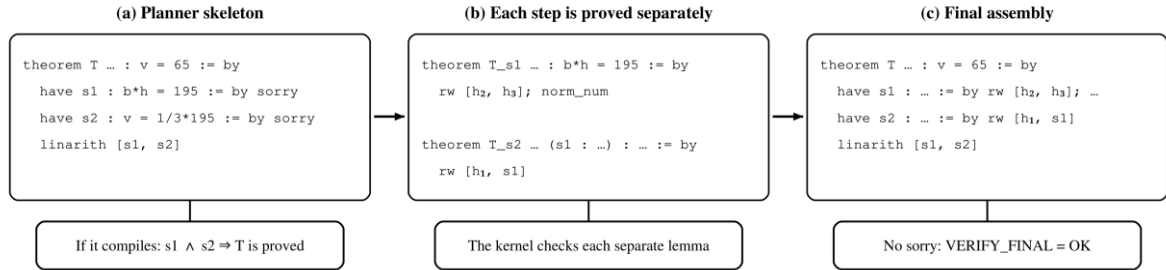


Fig. 4. From skeleton to proof (mathd_algebra_478): (a) steps with sorry, the first part of Eq. 2; (b) each step as a separate lemma, the second part; (c) everything put together with no sorry, Eq. 1.

Equation 3: cosine similarity. Used in the Retriever; $e(\cdot)$ is the BGE-M3 encoder, g the step and ℓ a lemma.

$$\text{sim}(g, \ell) = \frac{\langle e(g), e(\ell) \rangle}{\|e(g)\| \|e(\ell)\|} \quad (3)$$

In plain words: it measures how similar the step and the lemma are, 1 if they point the same way and 0 if they have nothing in common. It compares the direction of the vectors, which is what shows if two texts mean something similar.

Equation 4: top-k retrieval. Used in the Retriever; $\mathcal{M}$ are the 179,269 lemmas.

$$R_k(g) = \arg \max_{S \subset \mathcal{M}, |S|=k} \sum_{\ell \in S} \text{sim}(g, \ell) \quad (4)$$

In plain words: it returns the k lemmas that are most similar to the step. Since the vectors are normalized, FAISS (IndexFlatIP) solves it exactly.

Equation 5: Jaccard index. Used in the Generator; T_i are the tactics suggested by sketch i .

$$J(T_i, T_j) = \frac{|T_i \cap T_j|}{|T_i \cup T_j|} \quad (5)$$

In plain words: the share of tactics that two sketches have in common. We compare tactics and not words, because two sketches can say the same thing with other words.

Equation 6: consistency vote. Used in the Generator (2 sketches, 1 in retries), based on [10].

$$i^* = \arg \max_i \sum_{j \neq i} J(T_i, T_j), \quad a = \frac{1}{n} \sum_j \mathbb{1} [J(T_{i^*}, T_j) \geq 0.5] \quad (6)$$

In plain words: the sketch that looks most like the others wins, and a is the share of sketches that look like it. If several samples agree, that path is more likely to be the right one.

Equation 7: recall@k. Used to evaluate the Retriever with $N = 1,000$ theorems; $\text{Rel}(t)$ are the lemmas the proof of t uses.

$$\text{recall@k} = \frac{1}{N} \sum_{t=1}^N \frac{|\text{Rel}(t) \cap R_k(t)|}{|\text{Rel}(t)|} \quad (7)$$

In plain words: out of the lemmas that were really needed, how many show up among the k it returns, on average. We use it to choose k .

Equation 8: budget limit. Applied by the function that calls the models; $c_a(p)$ are the calls of agent a on problem p .

$$\sum_{a \in \mathcal{A}} c_a(p) \leq 20 \quad (8)$$

In plain words: all the agents together cannot make more than 20 calls per problem. It is a hard limit: if it is passed, the program stops.

Equation 9: solve rate. It is the main metric of the ablation (Table V).

$$\text{SR} = \frac{1}{N} \sum_{p=1}^N \text{solved}(p) \quad (9)$$

In plain words: the percentage of problems the kernel accepted.

Equation 10: false positive rate. Used in baseline B, where the model judges its own proof.

$$\text{FP} = \frac{|\{p : \text{self_eval}(p) = \text{correct} \wedge \text{solved}(p) = 0\}|}{|\{p : \text{self_eval}(p) = \text{correct}\}|} \quad (10)$$

In plain words: out of the proofs the model said were fine, how many the kernel rejected. It is the third contribution of the project.

Equation 11: Wilson interval. We will use it in week 12; $\hat{p}$ is the observed rate, N the number of problems and $z = 1.96$.

$$\frac{\hat{p} + \frac{z^2}{2N} \pm z \sqrt{\frac{\hat{p}(1 - \hat{p})}{N} + \frac{z^2}{4N^2}}}{1 + \frac{z^2}{N}} \quad (11)$$

In plain words: the range where the real rate most likely is. With few problems it does not give limits outside $[0, 1]$, which can happen with the normal interval.

TABLE IV
SUMMARY OF THE MATH MODEL

Eq.	Name	Where it is used	What for
1	success criterion	VERIFY_FINAL	only the kernel declares success
2	skeleton rule	VERIFY_SKELETON	check the plan before proving
3–4	cosine and top-k	Retriever	choose the k lemmas
5–6	Jaccard and vote	Generator	choose the sketch
7	recall@k	Retriever evaluation	choose k
8	budget	all calls	limit of 20 calls
9–10	solve rate and false positives	ablation, baseline B	main metrics
11	Wilson interval	analysis (week 12)	uncertainty of the rates

H. Local language models

At first the design used a big model through an API, which charges USD 5 per million input tokens and USD 25 per million output tokens. To avoid that cost we moved the Planner, the Generator and the Critic to Qwen3-4B [19] running locally with Ollama [20], and the only thing that changed was the module that talks to the models. We use the *instruct* version, with thinking turned off, an 8K context and output forced to JSON (see IV-D). What we lose is quality, a 4B model plans worse, so our rates cannot be compared with systems that use API models, but the ablation is still useful because all the configurations use the same models. The Autoformalizer still uses Goedel-Prover-V2-8B, which does not fully fit in 4 GB, so we have to serve it with vLLM on a borrowed GPU or with Ollama split between GPU and CPU.

I. Experimental design

TABLE V
ABLATION CONFIGURATIONS AND STATUS

#	Configuration	Status at week 8
1	Baseline A: the prover, one call, full proof	code ready
2	Baseline B: natural language reasoning + self-evaluation	code ready
3	Multi-agent without Retriever and without Critic	code ready, tested with the kernel
4	Multi-agent with Retriever, without Critic	week 9
5	Full system	week 9

The checkpoint for weeks 5 and 6 asks that skeletons compile in more than 70 % of the development set. **We set that 70 % ourselves, it does not come from the literature.** The reason is that if the skeleton does not compile the problem is lost anyway, so that rate is a ceiling for the solve rate, and since the goal for the full system is to solve between 30 % and 60 %, the Planner has to be well above that so it is not the bottleneck.

IV. RESULTS AND DISCUSSION

The parts that depend on the models were tested with answers written by hand, and the parts that depend on Lean were tested against the real kernel, so there are no solve rates yet.

TABLE VI
CHECKPOINTS PER WEEK

Weeks	Checkpoint	Result
1–2	verify() tells the verdicts apart; 2+2=4 from Python	done
3–4	baselines run on development with cost per problem	code ready, run missing
5–6	skeletons compile in >70 % of development; recall@k curve	curve ready, rate missing
7–8	a problem goes in and a kernel verdict comes out	done with a simulated model

A. Environment and Verifier

All 244 development problems compile with our Mathlib version. We have 38 tests without Lean and 18 against the real kernel, and all of them pass. We closed the week 7–8 checkpoint with mathd_algebra_48: the Planner splits it into one step, the kernel accepts the skeleton, the step is proved as a separate lemma and in the end the kernel accepts the full theorem:

```

theorem mathd_algebra_48 (q e : ℂ) (h₀ : q = 9 - 4 * Complex.I)
  (h₁ : e = -3 - 4 * Complex.I) : q - e = 12 := by
  have step1 : q - e = (9 - 4 * Complex.I) - (-3 - 4 * Complex.I) := by
    rw [h₀, h₁]
  rw [step1]
  ring

```

We also tested the opposite case: a bad proof two times in a row ends in `UNSOLVED_GOALS`, and the budget stops the graph when the limit is passed.

B. Bugs that only showed up with real Lean

With REPL answers written by hand everything passed, but when we connected real Lean six bugs showed up (Table VII). The ones that worried us most were that the REPL loaded an empty environment without any warning, and that Lean 3 syntax was reported as a missing lemma. What we learned is that made-up answers repeat what you think the system does, not what it really does. We also saw that in practice the 120 s wall clock limit is always the one that stops things, and that 2 or 3 REPLs fit in parallel in the 8 GB of Docker.

TABLE VII
BUGS FOUND WHEN TESTING AGAINST REAL LEAN

Bug	What happened	Fix
REPL without <code>lake env</code>	<code>import Mathlib</code> loaded nothing	start the REPL with <code>lake env</code>
JSON with escaped Unicode	$\mathcal{N}$ gave a syntax error	send UTF-8 without escaping
Lean 3 <code>begin ... end</code>	reported as missing lemma	classified as syntax
Lean dies from stack overflow	counted as syntax	counts as TIMEOUT
Orphan REPL	still alive with ~3 GB after a timeout	kill the whole process group
Step different from skeleton	JSON and skeleton did not match	take the text from the skeleton

C. Retriever

Extracting the corpus took 23 minutes and building the index took 34. Fig. 5 shows `recall@k` over 1,000 theorems (Eq. 7). The numbers are low and we leave them like that. When the query has names, the exact lemma comes first, but when it is only symbols ($0 \leq x \wedge 2$) obvious lemmas like `sq_nonneg` do not even appear in the top five, because the encoder relies a lot on names, and the reference of which lemmas are relevant is also noisy. Because of this, a reasonable `k` is between 10 and 20.

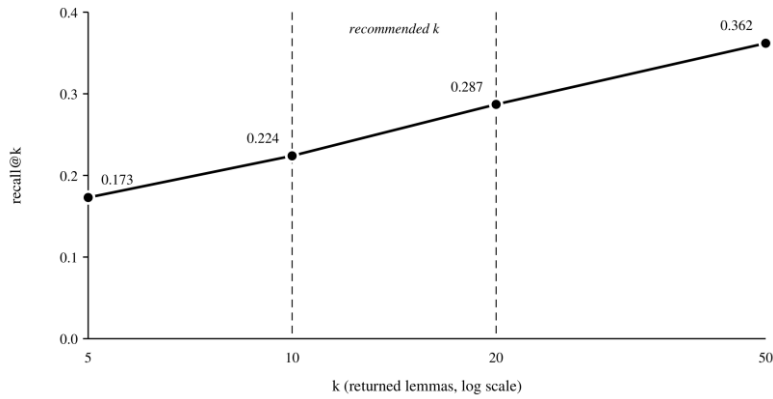


Fig. 5. Recall@k curve of the Retriever ($n = 1,000$, Eq. 7). The dashed lines mark the range of `k` we use.

D. First test of the local models

TABLE VIII
SMOKE TEST OF THE PLANNER WITH LOCAL MODELS

Model	Configuration	Result
llama3.2 (3B)	default	78 s, 97 tokens; broken JSON (bad escapes) and Lean that makes no sense
qwen3:4b	thinking on, 16K context	1,720 s (≈ 29 min); spent all 8,192 tokens thinking and gave no answer
qwen3:4b	thinking off, 8K	kept reasoning inside the text and ran out of its 4,096 tokens before any JSON
qwen3:4b-instruct	thinking off, 8K, forced JSON	26 s, 278 tokens; valid JSON with all the contract fields, but the skeleton is missing the final tactic

The connection works and the cost per call is zero. The thinking version is not usable on this GPU, it uses all its tokens before answering. The *instruct* version with forced JSON answered in 26 s, but the skeleton it gave does not have the tactic that closes the theorem, so `VERIFY_SKELETON` would reject it. The kernel catches the error, which is exactly its job, but the skeleton rate might end up below 70 % and we may need to adjust the Planner prompt.

V. CONCLUSIONS

By week 8 the system already runs end to end, and the real kernel made a decision on a miniF2F problem, with the statement checked, the skeleton verified before proving and the full proof verified at the end. The Verifier is deterministic and it is the only one that decides, and the Critic is already designed and comes in week 9. Testing against the real kernel was necessary, six bugs showed up that would have given us false results. The Retriever works, but its recall is low with queries that are only symbols. With local models we stopped paying per call, but we lose time and quality in exchange. What is still missing are the solve rates.

VI. FUTURE WORK

- **Local run:** the baselines and `plan_only` on development with Ollama and the prover served with vLLM.
- **Week 9:** add the Critic and the Retriever to the graph, and also query the Retriever in natural language.
- **Week 10:** build the test set of 40 to 60 problems, add PutnamBench and freeze the prompts.
- **Weeks 11–13:** run the experiment, analyze with Wilson (Eq. 11), McNemar and bootstrap, and write the final report.

REFERENCES

- [1] L. de Moura and S. Ullrich, “The Lean 4 theorem prover and programming language,” in *Automated Deduction – CADE 28*, 2021, pp. 625–635.
- [2] The mathlib Community, “The Lean mathematical library,” in *Proc. 9th ACM SIGPLAN Int. Conf. on Certified Programs and Proofs (CPP)*, 2020, pp. 367–381.
- [3] K. Zheng, J. M. Han, and S. Polu, “miniF2F: a cross-system benchmark for formal Olympiad-level mathematics,” in *ICLR*, 2022.
- [4] A. Q. Jiang et al., “Draft, sketch, and prove: Guiding formal theorem provers with informal proofs,” in *ICLR*, 2023.
- [5] K. Yang et al., “LeanDojo: Theorem proving with retrieval-augmented language models,” in *NeurIPS, Datasets and Benchmarks Track*, 2023.
- [6] E. First, M. N. Rabe, T. Ringer, and Y. Brun, “Baldur: Whole-proof generation and repair with large language models,” in *Proc. ACM ESEC/FSE*, 2023.
- [7] H. Xin et al., “DeepSeek-Prover-V1.5: Harnessing proof assistant feedback for reinforcement learning and Monte-Carlo tree search,” *arXiv preprint*, 2024.
- [8] Y. Lin et al., “Goedel-Prover-V2: Scaling formal theorem proving with scaffolded data synthesis and self-correction,” *arXiv preprint*, 2025.
- [9] J. Wei et al., “Chain-of-thought prompting elicits reasoning in large language models,” in *NeurIPS*, 2022.
- [10] X. Wang et al., “Self-consistency improves chain of thought reasoning in language models,” in *ICLR*, 2023.
- [11] S. Yao et al., “ReAct: Synergizing reasoning and acting in language models,” in *ICLR*, 2023.
- [12] N. Shinn et al., “Reflexion: Language agents with verbal reinforcement learning,” in *NeurIPS*, 2023.
- [13] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *NeurIPS*, vol. 33, 2020, pp. 9459–9474.
- [14] V. Karpukhin et al., “Dense passage retrieval for open-domain question answering,” in *Proc. EMNLP*, 2020, pp. 6769–6781.
- [15] J. Chen et al., “M3-Embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation,” in *Findings of ACL*, 2024.

- [16] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [17] Y. Wu et al., “Autoformalization with large language models,” in *NeurIPS*, 2022.
- [18] G. Tsoukalas et al., “PutnamBench: Evaluating neural theorem-provers on the Putnam mathematical competition,” in *NeurIPS, Datasets and Benchmarks Track*, 2024.
- [19] A. Yang et al., “Qwen3 technical report,” *arXiv preprint arXiv:2505.09388*, 2025.
- [20] Ollama, “Ollama: get up and running with large language models locally,” software, 2026. [Online]. Available: <https://ollama.com>
- [21] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.